# HIGH-LEVEL DESIGN (HLD)

# Title: RAG-Based Customer Support Assistant

**Submitted by-** Manjushree M

**ID:** IN126054202

**Organization:** Innomatics Research Labs

**Date:** 23/04/2026

# 1. System Overview

## Problem Definition

The system aims to build **a RAG-based Customer Support Assistant** that can:

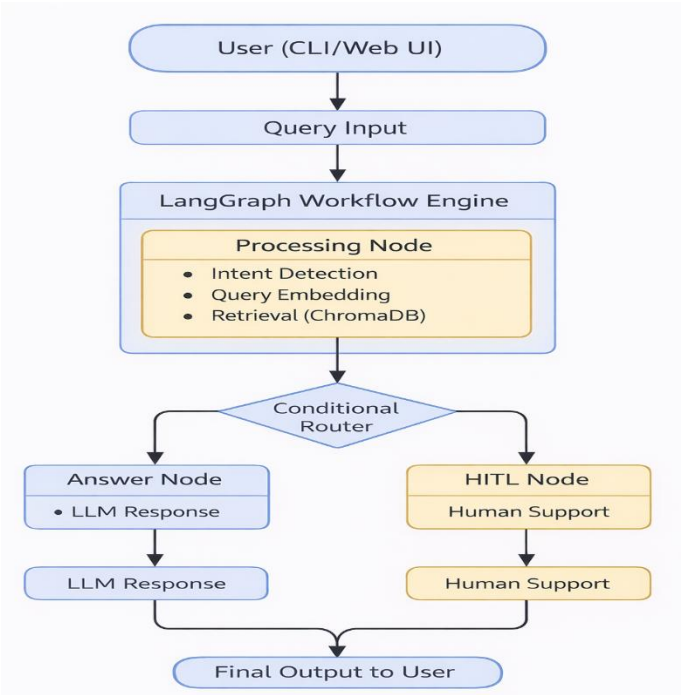
- Understand user queries
- Retrieve relevant information from a PDF knowledge base
- Generate contextual answers
- Escalate to human support when needed

## Scope

- Input: User query (CLI/Web)
- Output: AI-generated response or human escalation
- Supports:
    - PDF ingestion
    - Context-aware answering
    - Workflow-based routing
    - HITL escalation

---

# 2. Architecture Diagram




---

# 3. Component Description

### 1. Document Loader

Loads PDF and extracts raw text.

### 2. Chunking Strategy

Splits text into smaller chunks (e.g., 500 tokens with overlap).

### 3. Embedding Model

Converts text chunks into vector representations.

### 4. Vector Store (ChromaDB)

Stores embeddings for similarity search.

### 5. Retriever

Fetches relevant chunks based on user query.

### 6. LLM

Generates contextual responses using retrieved data.

### 7. Graph Workflow Engine (LangGraph)

Controls flow:

- Input → Process → Output

**8. Routing Layer**

Decides:

- Answer directly
- Escalate to human

**9. HITL Module**

Handles escalation to human agents.

---

## 4. Data Flow

**Document Flow**

PDF → Load → Chunk → Embed → Store in ChromaDB

**Query Flow**

User Query → Embed Query → Retrieve Chunks → LLM → Response
→ (if low confidence) → HITL

---

## 5. Technology Choices

- **ChromaDB** → Lightweight, easy vector storage
- **LangGraph** → Workflow control + state management
- **LLM (OpenAI / Llama)** → Natural language understanding
- **Python + LangChain** → Integration layer

---

## 6. Scalability Considerations

- Large PDFs → Chunking + batching
- High query load → Caching + async processing
- Latency → Optimize retrieval + smaller prompts

---